

1 Carbon-Aware Scheduling: MILP Formulation

1.1 Problem Definition

We formulate the carbon-aware scheduling problem as a Mixed Integer Linear Programming (MILP) problem. The objective is to find an optimal assignment of pods (computational tasks) to nodes (computing resources) and timeslots while minimizing the total carbon emissions.

1.2 Sets and Indices

$$i \in \mathcal{P} \quad \text{Set of pods} \quad (1)$$

$$j \in \mathcal{F} \quad \text{Set of nodes} \quad (2)$$

$$t \in \mathcal{T} \quad \text{Set of timeslots} \quad (3)$$

1.3 Parameters

$$D_i \in \mathbb{N}^+ \quad \text{Deadline timeslot for pod } i \quad (4)$$

$$s_i \in \mathbb{N}^+ \quad \text{Earliest timeslot for pod } i \quad (5)$$

$$d_i \in \mathbb{N}^+ \quad \text{Duration of pod } i \text{ (in timeslots)} \quad (6)$$

$$w_i \in \mathbb{N}^+ \quad \text{Temporal slack for pod } i \quad (7)$$

$$T_{max} \in \mathbb{N}^+ \quad \text{Maximum available timeslot index} \quad (8)$$

$$c_i \in \mathbb{R}^+ \quad \text{CPU request of pod } i \quad (9)$$

$$r_i \in \mathbb{R}^+ \quad \text{RAM request of pod } i \quad (10)$$

$$C_{jt} \in \mathbb{R}^+ \quad \text{Available CPU of node } j \text{ in timeslot } t \quad (11)$$

$$R_{jt} \in \mathbb{R}^+ \quad \text{Available RAM of node } j \text{ in timeslot } t \quad (12)$$

$$p_{ij} \in \mathbb{R}^+ \quad \text{Power consumption of pod } i \text{ on node } j \quad (13)$$

$$CI_{j,t} \in \mathbb{R}^+ \quad \text{Carbon intensity of node } j \text{ in timeslot } t \quad (14)$$

$$EC_j \in \mathbb{R}^+ \quad \text{Embodied carbon of node } j \quad (15)$$

$$L_j \in \mathbb{R}^+ \quad \text{Expected lifetime of node } j \quad (16)$$

$$u_j \in \mathbb{R}^+ \quad \text{Utilization factor of node } j \quad (17)$$

$$e_{ijt} \in \mathbb{R}^+ \quad \text{Carbon emissions of pod } i \text{ on node } j \text{ at timeslot } t \quad (18)$$

The carbon emissions e_{ijt} are calculated considering:

- Operational carbon: Based on the pod's resource consumption, node's power profile, and regional carbon intensity forecasts across the pod's execution duration
- Embodied carbon: Accounting for the amortized manufacturing emissions of the hardware over its lifetime

1.4 Decision Variables

$$x_{ijt} = \begin{cases} 1 & \text{if pod } i \text{ is placed on node } j \text{ starting at timeslot } t \\ 0 & \text{otherwise} \end{cases} \quad (19)$$

1.5 Feasible Placements

Let $\mathcal{V}_i \subset \mathcal{F} \times \mathcal{T}$ be the set of valid node-timeslot pairs for pod i :

$$\mathcal{V}_i = \{(j, t) \in \mathcal{F} \times \mathcal{T} \mid t \geq s_i \text{ and} \quad (20)$$

$$t + d_i \leq D_i \text{ and} \quad (21)$$

$$C_{j(t+k)} \geq c_i \text{ for all } k \in \{0, 1, \dots, d_i - 1\} \text{ and} \quad (22)$$

$$R_{j(t+k)} \geq r_i \text{ for all } k \in \{0, 1, \dots, d_i - 1\}\} \quad (23)$$

The deadline timeslot D_i for pod i is calculated as:

$$D_i = s_i + d_i + w_i \leq T_{max} \quad (24)$$

where the deadline includes the pod's execution duration and temporal slack for scheduling flexibility.

1.6 Mathematical Model

1.6.1 Objective Function

Minimize total carbon emissions:

$$\text{Minimize } Z = \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} e_{ijt} \cdot x_{ijt} \quad (25)$$

1.6.2 Constraints

Assignment constraint: Each pod must be placed exactly once:

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} = 1 \quad \forall i \in \mathcal{P} \quad (26)$$

CPU capacity constraints: The total CPU usage in each node and timeslot must not exceed capacity:

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} c_i \cdot x_{ijt'} \leq C_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \quad (27)$$

where $\mathcal{T}_{it} = \{t' \in \mathcal{T} \mid t' \leq t < t' + d_i, (j, t') \in \mathcal{V}_i\}$ represents the set of starting timeslots t' such that pod i would be running at timeslot t if started at t' .

RAM capacity constraints: The total RAM usage in each node and timeslot must not exceed capacity:

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} r_i \cdot x_{ijt'} \leq R_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \quad (28)$$

Binary constraint: Decision variables are binary:

$$x_{ijt} \in \{0, 1\} \quad \forall i \in \mathcal{P}, \forall (j, t) \in \mathcal{V}_i \quad (29)$$

1.7 Emissions Calculation

The carbon emissions parameter e_{ijt} for running pod i on node j starting at timeslot t is calculated as:

$$e_{ijt} = \sum_{k=0}^{d_i-1} (op_{ijt+k} + emb_{ij}) \quad (30)$$

where:

- op_{ijt+k} is the operational carbon emissions in timeslot $t+k$:

$$op_{ijt+k} = p_{ij} \cdot CI_{j,t+k} \quad (31)$$

with p_{ij} being the power consumption of pod i on node j and $CI_{j,t+k}$ being the carbon intensity forecast for the region of node j in timeslot $t+k$.

- emb_{ij} is the embodied carbon emissions attributed to pod i on node j for one timeslot:

$$emb_{ij} = \frac{EC_j}{L_j \cdot u_j} \quad (32)$$

with EC_j being the total embodied carbon of node j , L_j being the node's expected lifetime in hours, and u_j representing the pod's resource usage as a fraction of the node's total capacity.

1.8 Relaxation Methods for Infeasible MILP Instances

The carbon-aware scheduling MILP formulation may become infeasible when resource constraints are over-subscribed or when pod deadlines are incompatible with available scheduling windows. To address this fundamental challenge, we develop a hierarchy of relaxation techniques that systematically transform the problem to ensure feasibility while preserving solution quality.

1.8.1 Theoretical Foundation

Let $\mathcal{M}$ denote the original MILP formulation. The feasibility of $\mathcal{M}$ depends on the existence of a solution $\mathbf{x} \in \{0, 1\}^{|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}|}$ satisfying constraints (14)-(17). When $\mathcal{M}$ is infeasible, we seek to solve a relaxed problem $\mathcal{M}'$ that:

1. Guarantees feasibility: $\mathcal{M}'$ always admits a solution
2. Maximizes pod placement: Priority is given to scheduling as many pods as possible
3. Minimizes carbon emissions: Among solutions with equal placement counts, prefer lower emissions
4. Maintains computational tractability: Relaxation overhead should be minimal

1.8.2 Multi-Objective Optimization with Penalty Functions

Our primary relaxation strategy transforms the hard assignment constraints into soft constraints using penalty functions. This approach, known as *goal programming* in the operations research literature, converts the infeasible MILP into a bi-objective optimization problem.

Extended Decision Variables: We introduce slack variables to capture unplaced pods:

$$y_i \in \{0, 1\} \quad \forall i \in \mathcal{P} \quad (33)$$

where $y_i = 1$ indicates that pod i remains unplaced.

Reformulated Objective Function: The bi-objective formulation becomes:

$$\text{Minimize} \quad Z(\mathbf{x}, \mathbf{y}) = \lambda \sum_{i \in \mathcal{P}} y_i + \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} e_{ijt} \cdot x_{ijt} \quad (34)$$

where $\lambda \gg \max_{i,j,t} e_{ijt}$ is a penalty weight ensuring lexicographic optimization priority.

Softened Assignment Constraints: The hard assignment constraint (14) is replaced with:

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} + y_i = 1 \quad \forall i \in \mathcal{P} \quad (35)$$

This constraint ensures that each pod is either placed exactly once ($\sum x_{ijt} = 1, y_i = 0$) or marked as unplaced ($\sum x_{ijt} = 0, y_i = 1$).

Theoretical Properties:

Theorem 1.1 (Feasibility Guarantee). *The relaxed MILP formulation $\mathcal{M}'$ with objective (20) and constraints (21), (15)-(17) is always feasible.*

Proof. The solution $\mathbf{x} = \mathbf{0}$ and $y_i = 1$ for all $i \in \mathcal{P}$ satisfies all constraints trivially, as no resources are consumed when all pods are unplaced. $\square$

Theorem 1.2 (Optimality Preservation). *For sufficiently large penalty λ , if the original problem $\mathcal{M}$ is feasible, then the optimal solution to $\mathcal{M}'$ places all pods (i.e., $\mathbf{y} = \mathbf{0}$) and achieves the same objective value as $\mathcal{M}$.*

Proof. If $\mathcal{M}$ is feasible with optimal value Z^* , then $\mathcal{M}'$ has a feasible solution with objective value Z^* . Any solution with unplaced pods has objective value at least $\lambda + Z^* > Z^*$ for $\lambda > 0$, thus the optimal solution to $\mathcal{M}'$ must place all pods. $\square$

1.8.3 Iterative Constraint Relaxation

When the multi-objective approach encounters solver limitations or remains computationally intractable, we apply an iterative constraint relaxation strategy that systematically reduces problem complexity.

Constraint Pressure Quantification: We define a constraint pressure metric ϕ_i for each pod i :

$$\phi_i = \alpha \cdot \frac{1}{|\mathcal{V}_i|} + \beta \cdot \max\left(0, \frac{\tau_{max} - \tau_i}{\tau_{max}}\right) \quad (36)$$

where:

- $|\mathcal{V}_i|$ is the number of feasible placements for pod i
- $\tau_i = D_i - s_i - d_i$ is the scheduling flexibility (slack) for pod i
- $\tau_{max} = 5$ is the maximum considered slack threshold
- $\alpha = 10$ and $\beta = 1$ are empirically determined weights

Iterative Relaxation Algorithm:

Algorithm 1 Iterative Constraint Relaxation

- 1: **Input:** Pod set $\mathcal{P}$, maximum iterations k_{max} , removal fraction $\rho = 0.1$
 - 2: **Initialize:** $\mathcal{P}^{(0)} = \mathcal{P}$, $\mathcal{P}^{removed} = \emptyset$, $k = 0$
 - 3: **while** $|\mathcal{P}^{(k)}| > 0$ and $k < k_{max}$ **do**
 - 4: Solve MILP $\mathcal{M}(\mathcal{P}^{(k)})$ with pod set $\mathcal{P}^{(k)}$
 - 5: **if** $\mathcal{M}(\mathcal{P}^{(k)})$ is feasible **then**
 - 6: **return** optimal solution and $\mathcal{P}^{removed}$
 - 7: **end if**
 - 8: Compute constraint pressure ϕ_i for all $i \in \mathcal{P}^{(k)}$
 - 9: $n_{remove} = \max(1, \min(20, \lceil \rho \cdot |\mathcal{P}^{(k)}| \rceil))$
 - 10: $\mathcal{P}^{remove} = \{i \in \mathcal{P}^{(k)} \mid \phi_i \text{ among top } n_{remove} \text{ values}\}$
 - 11: $\mathcal{P}^{(k+1)} = \mathcal{P}^{(k)} \setminus \mathcal{P}^{remove}$
 - 12: $\mathcal{P}^{removed} = \mathcal{P}^{removed} \cup \mathcal{P}^{remove}$
 - 13: $k = k + 1$
 - 14: **end while**
 - 15: **return** empty solution and $\mathcal{P}^{removed} = \mathcal{P}$
-

Greedy Recovery Phase: For pods in $\mathcal{P}^{removed}$, we attempt placement using a greedy heuristic that selects the feasible placement with minimum carbon emissions:

$$(j^*, t^*) = \arg \min_{(j,t) \in \mathcal{V}_i \cap \mathcal{A}} e_{ijt} \quad (37)$$

where $\mathcal{A}$ represents the set of currently available node-timeslot pairs.

1.8.4 Complexity Analysis

Multi-Objective Approach: The relaxed MILP has $|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}| + |\mathcal{P}|$ binary variables and $|\mathcal{P}| + |\mathcal{F}| \times |\mathcal{T}| \times 2$ constraints. The theoretical complexity remains NP-hard, but practical solving time may improve due to relaxed constraints.

Iterative Relaxation: In the worst case, the algorithm requires $O(|\mathcal{P}|/\rho)$ iterations, each solving a MILP with decreasing size. The overall complexity is $O(|\mathcal{P}|^2 \cdot T_{solve})$ where T_{solve} is the time to solve a single MILP instance.

Greedy Recovery: The greedy phase has complexity $O(|\mathcal{P}^{removed}| \times |\mathcal{F}| \times |\mathcal{T}|)$, which is polynomial in the problem size.

1.8.5 Implementation Considerations

The relaxation methods are implemented in a hierarchical manner:

1. **Primary approach:** Multi-objective optimization with penalty functions
2. **Fallback 1:** Iterative constraint relaxation when multi-objective fails
3. **Fallback 2:** Greedy placement heuristic for remaining unplaced pods

The penalty weight λ is a critical parameter that balances placement priority against emissions optimization. In practice, values of $\lambda = 10^4$ have been observed to provide reasonable trade-offs, though systematic parameter tuning may be required for different problem instances.

1.9 Implementation Details

In practice, the problem is solved using the PuLP library with the following approach:

1. Determine the earliest timeslot (s_i) for each pod based on its source file
2. Precompute all valid placements for each pod, respecting the earliest timeslot constraint
3. Create binary decision variables for each valid placement
4. Define the objective function to minimize total carbon emissions
5. Add constraints ensuring each pod is placed exactly once

6. Add resource capacity constraints for CPU and RAM
7. Solve the MILP problem using the CBC solver with a time limit
8. Extract the solution from the decision variables